



Shortest Unique Palindromic Substring Queries on Run-Length Encoded Strings

Kiichi Watanabe^{1(✉)}, Yuto Nakashima², Shunsuke Inenaga², Hideo Bannai²,
and Masayuki Takeda²

¹ Department of Electrical Engineering and Computer Science,
Kyushu University, Fukuoka, Japan

watanabe.kiichi.849@s.kyushu-u.ac.jp

² Department of Informatics, Kyushu University, Fukuoka, Japan
{yuto.nakashima, inenaga, bannai, takeda}@inf.kyushu-u.ac.jp

Abstract. For a string S , a palindromic substring $S[i..j]$ is said to be a *shortest unique palindromic substring (SUPS)* for an interval $[s, t]$ in S , if $S[i..j]$ occurs exactly once in S , the interval $[i, j]$ contains $[s, t]$, and every palindromic substring containing $[s, t]$ which is shorter than $S[i..j]$ occurs at least twice in S . In this paper, we study the problem of answering SUPS queries on run-length encoded strings. We show how to preprocess a given run-length encoded string RLE_S of size m in $O(m)$ space and $O(m \log \sigma_{RLE_S} + m\sqrt{\log m / \log \log m})$ time so that all SUPSs for any subsequent query interval can be answered in $O(\sqrt{\log m / \log \log m} + \alpha)$ time, where α is the number of outputs, and σ_{RLE_S} is the number of distinct runs of RLE_S .

1 Introduction

The *shortest unique substring (SUS)* problem, which is formalized below, is a recent trend in the string processing community. Consider a string S of length n . A substring $X = S[i..j]$ of S is called a SUS for a position p ($1 \leq p \leq n$) iff the interval $[i..j]$ contains p , X occurs in S exactly once, and every substring containing p which is shorter than $S[i..j]$ occurs at least twice in S . The SUS problem is to preprocess a given string S so that SUSs for query positions p can be answered quickly. The study on the SUS problem was initiated by Pei et al., and is motivated by an application to bioinformatics e.g., designing polymerase chain reaction (PCR) primer [11]. Pei et al. [11] showed an $\Theta(n^2)$ -time and space preprocessing scheme such that all k SUSs for a query position can be answered in $O(k)$ time. Later, two independent groups, Tsuruta et al. [13] and Ileri et al. [7], showed algorithms that use $\Theta(n)$ time and space¹ for preprocessing, and all SUSs can be answered in $O(k)$ time per query. To be able to handle huge text data where n can be massively large, there have been further efforts to reduce

¹ Throughout this paper, we measure the space complexity of an algorithm with the number of *words* that the algorithm occupies in the word RAM model, unless otherwise stated.

the space usage. Hon et al. [5] proposed an “in-place” algorithm which works within space of the input string S and two output arrays A and B of length n each, namely, in $n \log_2 \sigma$ bits plus $2n$ words of space. After the execution of their algorithm that takes $O(n)$ time, the beginning and ending positions of a SUS for each text position i ($1 \leq i \leq n$) are consequently stored in $A[i]$ and $B[i]$, respectively, and S remains unchanged. Hon et al.’s algorithm can be extended to handle SUSs with approximate matches, with a penalty of $O(n^2)$ preprocessing time. For a pre-determined parameter τ , Ganguly et al. [4] proposed a time-space trade-off algorithm for the SUS problem that uses $O(n/\tau)$ additional working space (apart from the input string S) and answers each query in $O(n\tau^2 \log \frac{n}{\tau})$ time. They also proposed a “succinct” data structure of $4n + o(n)$ bits of space that can be built in $O(n \log n)$ time and can answer a SUS for each given query position in $O(1)$ time. Another approach to reduce the space requirement for the SUS problem is to work on a “compressed” representation of the string S . Mieno et al. [10] developed a data structure of $\Theta(m)$ space (or $\Theta(m \log n)$ bits of space) that answers all k SUSs for a given position in $O(\sqrt{\log m / \log \log m} + k)$ time, where m is the size of the *run length encoding (RLE)* of the input string S . This data structure can be constructed in $O(m \log m)$ time with $O(m)$ words of working space if the input string S is already compressed by RLE, or in $O(n + m \log m)$ time with $O(m)$ working space if the input string S is given without being compressed.

A generalized version of the SUS problem, called the *interval SUS* problem, is to answer SUSs that contain a query interval $[s, t]$ with $1 \leq s \leq t \leq n$. Hu et al. [6] proposed an optimal $\Theta(n)$ time and space algorithm to preprocess a given string S so that all k SUSs for a given query interval are reported in $O(k)$ time. Mieno et al.’s data structure [10] also can answer interval SUS queries with the same preprocessing time/space and query time as above.

Recently, a new variant of the SUS problem, called the *shortest unique palindromic substring (SUPS)* problem is considered [8]. A substring $P = S[i..j]$ is called a SUPS for an interval $[s, t]$ iff P occurs exactly once in S , $[s, t] \subseteq [i, j]$, and every palindromic substring of S which contains interval $[s, t]$ and is shorter than P occurs at least twice in S . The study on the SUPS problem is motivated by an application in molecular biology. Inoue et al. [8] showed how to preprocess a given string S of length n in $\Theta(n)$ time and space so that all α SUPSs (if any) for a given interval can be answered in $O(\alpha + 1)$ time². While this solution is optimal in terms of the length n of the input string, no space-economical solutions for the SUPS problem were known.

In this paper, we present the *first* space-economical solution to the SUPS problem based on RLE. The proposed algorithm computes a data structure of $\Theta(m)$ space that answers each SUPS query in $O(\sqrt{\log m / \log \log m} + \alpha)$ time. The most interesting part of our algorithm is how to preprocess a given RLE string of length m in $O(m(\log \sigma_{RLE_S} + \sqrt{\log m / \log \log m}))$ time, where σ_{RLE_S} is the number of distinct runs in the RLE of S . Note that $\sigma_{RLE} \leq m$ always holds. For this sake, we propose RLE versions of Manacher’s maximal palindrome

² It is possible that $\alpha = 0$ for some intervals.

algorithm [9] and Rubinchik and Shur's eertree data structure [12], which may be of independent interest. We remark that our preprocessing scheme is quite different from Mieno et al.'s method [10] for the SUS problem on RLE strings and Inoue et al.'s method [8] for the SUPS problem on plain strings.

2 Preliminaries

2.1 Strings

Let Σ be an ordered *alphabet* of size σ . An element of Σ^* is called a *string*. The length of a string S is denoted by $|S|$. The empty string ε is a string of length 0. For a string $S = XYZ$, X , Y and Z are called a *prefix*, *substring*, and *suffix* of S , respectively. The i -th character of a string S is denoted by $S[i]$, for $1 \leq i \leq |S|$. Let $S[i..j]$ denote the substring of S that begins at position i and ends at position j , for $1 \leq i \leq j \leq |S|$. For convenience, let $S[i..j] = \varepsilon$ for $i > j$.

For any string S , let $S^R = S[|S|] \cdots S[1]$ denote the reversed string of S . A string P is called a *palindrome* iff $P = P^R$. A substring $P = S[i..j]$ of a string S is called a *palindromic substring* iff P is a palindrome. For a palindromic substring $P = S[i..j]$, $\frac{i+j}{2}$ is called the *center* of P . A palindromic substring $P = S[i..j]$ is said to be a *maximal palindrome* of S , iff $S[i-1] \neq S[j+1]$, $i = 1$ or $j = |S|$. A suffix of string S that is a palindrome is called a *palindromic suffix* of S . Clearly any palindromic suffix of S is a maximal palindrome of S .

We will use the following lemma in the analysis of our algorithm.

Lemma 1 ([3]). *Any string of length k can contain at most $k + 1$ distinct palindromic substrings (including the empty string ε).*

2.2 MUPSS and SUPSs

For any strings X and S , let $occ_S(X)$ denote the number of occurrences of X in S , i.e., $occ_S(X) = |\{i \mid S[i..i + |X| - 1] = X\}|$. A string X is called a *unique substring* of a string S iff $occ_S(X) = 1$. A substring $P = S[i..j]$ of string S is called a *minimal unique palindromic substring (MUPS)* of a string S iff (1) P is a unique palindromic substring of S and (2) either $|P| \geq 3$ and the palindrome $Q = S[i+1..j-1]$ satisfies $occ_S(Q) \geq 2$, or $1 \leq |P| \leq 2$.

Lemma 2 ([8]). *MUPSS do not nest, namely, for any pair of distinct MUPSS, one cannot contain the other.*

Due to Lemma 2, both of the beginning positions and the ending positions of MUPSS are monotonically increasing. Let $\mathcal{M}_S$ denote the list of MUPSS in S sorted in increasing order of their beginning positions (or equivalently the ending positions) in S .

Let $[s, t]$ be an integer interval over the positions in a string S , where $1 \leq s \leq t \leq |S|$. A substring $P = S[i..j]$ of string S is called a *shortest unique palindromic substring (SUPS)* for interval $[s, t]$ of S , iff (1) P is a unique palindromic substring of S , (2) $[s, t] \subseteq [i, j]$, and (3) there is no unique palindromic substring $Q = S[i'..j']$ such that $[s, t] \subseteq [i', j']$ and $|Q| < |P|$. We give an example in Fig. 1.

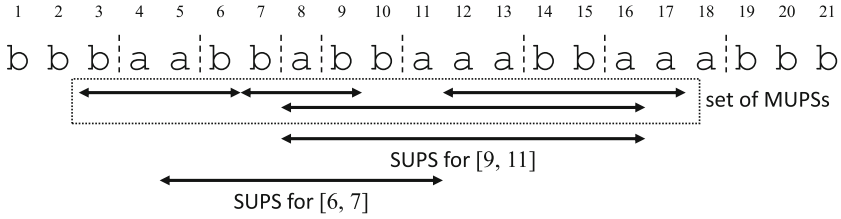


Fig. 1. This figure shows all *MUPs* and some *SUPSs* of string $RLE_S = b^3a^2b^2a^1b^2a^3b^2a^3b^3$. There are 4 *MUPs* illustrated in the box. The *SUPS* for interval $[6, 7]$ is $S[5..11]$, and the *SUPS* for interval $[9, 11]$ is $S[8..16]$.

2.3 Run Length Encoding (RLE)

The *run-length encoding* RLE_S of string S is a compact representation of S such that each maximal run of the same characters in S is represented by a pair of the character and the length of the run. More formally, let $\mathcal{N}$ denote the set of positive integers. For any non-empty string S , $RLE_S = (a_1, e_1), \dots, (a_m, e_m)$, where $a_j \in \Sigma$ and $e_j \in \mathcal{N}$ for any $1 \leq j \leq m$, and $a_j \neq a_{j+1}$ for any $1 \leq j < m$. E.g., if $S = aacccccccbbabbbb$, then $RLE_S = (a, 2), (c, 7), (b, 2), (a, 1), (b, 4)$. Each (a, e) in RLE_S is called a (character) *run*, and e is called the exponent of this run. We also denote each run by a^e when it seems more convenient and intuitive. For example, we would write as (a, e) when it seems more convenient to treat it as a kind of character (called an RLE-character), and would write as a^e when it seems more convenient to treat it as a string consisting of e a 's.

The *size* of RLE_S is the number m of runs in RLE_S . Let $Rb[j]$ (resp. $Re[j]$) denote the beginning (resp. ending) position of the j th run in the string S , i.e., $Rb[j] = 1 + \sum_{i=0}^{j-1} e_i$ with $e_0 = 0$ and $Re[j] = \sum_{i=1}^j e_i$. The *center* of the j th run is $\frac{Rb[j] + Re[j]}{2}$.

For any two ordered pairs $(a, e), (a', e') \in \Sigma \times \mathcal{N}$ of a character and positive integer, we define the equality such that $(a, e) = (a', e')$ iff $a = a'$ and $e = e'$ both hold. We also define a total order of these pairs such that $(a, e) < (a', e')$ iff $a < a'$, or $a = a'$ and $e < e'$.

An occurrence of a palindromic substring $P = S[i..i']$ of a string S with RLE_S of size m is said to be *RLE-bounded* if $i = Rb[j]$ and $i' = Re[j']$ for some $1 \leq j \leq j' \leq m$, namely, if both ends of the occurrence touch the boundaries of runs. An RLE-bounded occurrence $P = S[i..i']$ is said to be *RLE-maximal* if $(a_{j-1}, e_{j-1}) \neq (a_{j'+1}, e_{j'+1})$, $j = 1$ or $j' = m$. Note that an RLE-maximal occurrence of a palindrome may not be maximal in the string S . E.g., consider string $S = caabbcccbbaaaac$ with $RLE_S = c^1a^2b^2c^3b^2a^4c^1$.

- The occurrence of palindrome c^3 is RLE-bounded but is neither RLE-maximal nor maximal.
- The occurrence of palindrome $b^2c^3b^2$ is RLE-maximal but is not maximal.
- The occurrence of palindrome $a^2b^2c^3b^2a^2$ is not RLE-maximal but is maximal.

- The first (leftmost) occurrence of palindrome $\mathbf{a}^2$ is both RLE-maximal and maximal.

2.4 Problem

In what follows, we assume that our input strings are given as RLE strings. In this paper, we tackle the following problem.

Problem 1 (SUPS problem on run-length encoded strings).

Preprocess: $RLE_S = (a_1, e_1), \dots, (a_m, e_m)$ of size m representing a string S of length n .

Query: An integer interval $[s, t]$ ($1 \leq s \leq t \leq n$).

Return: All SUPSs for interval $[s, t]$.

In case the string S is given as a plain string of length n , then the time complexity of our algorithm will be increased by an additive factor of n that is needed to compute RLE_S , while the space usage will stay the same since RLE_S can be computed in constant space.

3 Computing MUPSs from RLE Strings

The following known lemma suggests that it is helpful to compute the set $\mathcal{M}_S$ of MUPSs of S as a preprocessing for the SUPS problem.

Lemma 3 ([8]). *For any SUPS $S[i..j]$ for some interval, there exists exactly one MUPS that is contained in the interval $[i, j]$. Furthermore, the MUPS has the same center as the SUPS $S[i..j]$.*

3.1 Properties of MUPSs on RLE Strings

Now we present some useful properties of MUPSs on the run-length encoded string $RLE_S = (a_1, e_1), \dots, (a_m, e_m)$.

Lemma 4. *For any MUPS $S[i..j]$ in S , there exists a unique integer k ($1 \leq k \leq m$) such that $\frac{i+j}{2} = \frac{Rb[k]+Re[k]}{2}$.*

Proof. Suppose on the contrary that there is a MUPS $S[i..j]$ such that $\frac{i+j}{2} \neq \frac{Rb[k]+Re[k]}{2}$ for any $1 \leq k \leq m$. Let l be the integer that satisfies $Rb[l] \leq \frac{i+j}{2} \leq Re[l]$. By the assumption, the longest palindrome whose center is $\frac{i+j}{2}$ is $a_l^{\min\{i-Rb[l]+1, Re[l]-j+1\}}$. However, this palindrome $a_l^{\min\{i-Rb[l]+1, Re[l]-j+1\}}$ occurs at least twice in the l th run $a_l^{e_l}$. Hence MUPS $S[i..j]$ is not a unique palindromic substring, a contradiction. $\square$

The following corollary is immediate from Lemma 4.

Corollary 1. *For any string S , $|\mathcal{M}_S| \leq m$.*

It is easy to see that the above bound is tight: for instance, any string where each run has a distinct character (i.e., $m = \sigma_{RLE}$) contains exactly m MUPSs. Our preprocessing and query algorithms which will follow are heavily dependent on this lemma and corollary.

3.2 RLE Version of Manacher's Algorithm

Due to Corollary 1, we can restrict ourselves to computing palindromic substrings whose center coincides with the center of each run. These palindromic substrings are called *run-centered* palindromes. Run-centered palindromes will be candidates of MUPSS of the string S .

To compute run-centered palindromes from RLE_S , we utilize Manacher's algorithm [9] that computes all maximal palindromes for a given (plain) string of length n in $O(n)$ time and space. Manacher's algorithm is based only on character equality comparisons, and hence it works with general alphabets.

Let us briefly recall how Manacher's algorithm works. It processes a given string S of length n from left to right. It computes an array **MaxPal** of length $2n + 1$ such that **MaxPal** $[c]$ stores the length of the maximal palindrome with center c for $c = 1, 1.5, 2, \dots, n - 1, n - 0.5, n$. Namely, Manacher's algorithm processes a given string S in an online manner from left to right. This algorithm is also able to compute, for each position $i = 1, \dots, n$, the longest palindromic suffix of $S[1..i]$ in an online manner.

Now we apply Manacher's algorithm to our run-length encoded input string $RLE_S = (a_1, e_1), \dots, (a_m, e_m)$. Then, what we obtain after the execution of Manacher's algorithm over RLE_S is all *RLE-maximal* palindromes of S . Note that by definition all RLE-maximal palindromes are run-centered. Since RLE_S can be regarded as a string of length m over an alphabet $\Sigma \times \mathcal{N}$, this takes $O(m)$ time and space.

Remark 1. If wanted, we can compute all *maximal* palindromes of S in $O(m)$ time after the execution of Manacher's algorithm to RLE_S . First, we compute every run-centered maximal palindrome P_l that has its center in each l th run in RLE_S . For each already computed run-centered RLE-maximal palindrome $Q_l = S[Rb[i]..Re[j]]$ with $1 < i \leq j < m$, it is enough to first check whether $a_{i-1} = a_{j+1}$. If no, then $P_l = Q_l$, and if yes then we can further extend both ends of Q_l with $(a_{i-1}, \min\{e_{i-1}, e_{j+1}\})$ and obtain P_l . As a side remark, we note that any other maximal palindromes of S are not run-centered, which means that any of them consists only of the same characters and lie inside of one character run. Such maximal palindromes are trivial and need not be explicitly computed.

3.3 RLE Version of Eertree Data Structure

The *eertree* [12] of a string S , denoted $eertree(S)$, is a pair of two rooted trees T_{odd} and T_{even} which represent all distinct palindromic substrings of S . The root of T_{odd} represents the empty string ε and each non-root node of T_{odd} represents a non-empty palindromic substring of S of odd length. Similarly, the root of T_{even} represents the empty string ε and each non-root node of T_{even} represents a non-empty palindromic substring of S of even length. From the root r of T_{odd} , there is a labeled directed edge (r, a, v) if v represents a single character $a \in \Sigma$. For any non-root node u of T_{odd} or T_{even} , there is a labeled directed edge (u, a, v) from u to node v with character label $a \in \Sigma$ if $aua = v$. For any node u , the labels of out-going edges of u must be mutually distinct.

By Lemma 1, any string S of length n can contain at most $n + 1$ distinct palindromic substrings (including the empty string ε). Thus, the size of $\text{eertree}(S)$ is linear in the string length n . Rubinchik and Shur [12] showed how to construct $\text{eertree}(S)$ in $O(n \log \sigma_S)$ time and $O(n)$ space, where σ_S is the number of distinct characters in S . They also showed how to compute the number of occurrences of each palindromic substring in $O(n \log \sigma_S)$ time and $O(n)$ space, using $\text{eertree}(S)$.

Now we introduce a new data structure named *RLE-eertrees* based on eertrees. Let $RLE_S = (a_1, e_1), \dots, (a_m, e_m)$, and let Σ_{RLE} be the set of maximal runs of S , namely, $\Sigma_{RLE} = \{(a, e) \mid (a, e) = (a_i, e_i) \text{ for some } 1 \leq i \leq m\}$. Let $\sigma_{RLE} = |\Sigma_{RLE}|$. Note that $\sigma_{RLE} \leq m$ always holds. The RLE-eertree of string S , denoted by $\text{e}^2\text{rtre}^2(S)$, is a *single eertree* T_{odd} over the RLE alphabet $\sigma_{RLE} \subset \Sigma \times \mathcal{N}$, which represents distinct run-centered palindromes of S which have an RLE-bounded occurrence $[i, i']$ such that $i = Rb[j]$ and $i' = Re[j']$ for some $1 \leq j \leq j' \leq m$ (namely, the both ends of the occurrence touch the boundary of runs), or an occurrence as a maximal palindrome in S . We remark that the number of runs in each palindromes in $\text{e}^2\text{rtre}^2(S)$ is odd, but their decompressed string length may be odd or even. In $\text{e}^2\text{rtre}^2(S)$, there is a directed labeled edge (u, a^e, v) from node u to node v with label $a^e \in \Sigma_{RLE}$ if (1) $a^e u a^e = v$, or (2) $u = \varepsilon$ and $v = a^e \in \Sigma \times \mathcal{N}$. Note that if the in-coming edge of a node u is labeled with a^e , then any out-going edge of u cannot have a label a^f with the same character a . Since $\text{e}^2\text{rtre}^2(S)$ is an eertree over the alphabet Σ_{RLE} of size $\sigma_{RLE} \leq m$, it is clear that the number of out-going edges of each node is bounded by σ_{RLE} . We give an example of $\text{e}^2\text{rtre}^2(S)$ in Fig. 2.

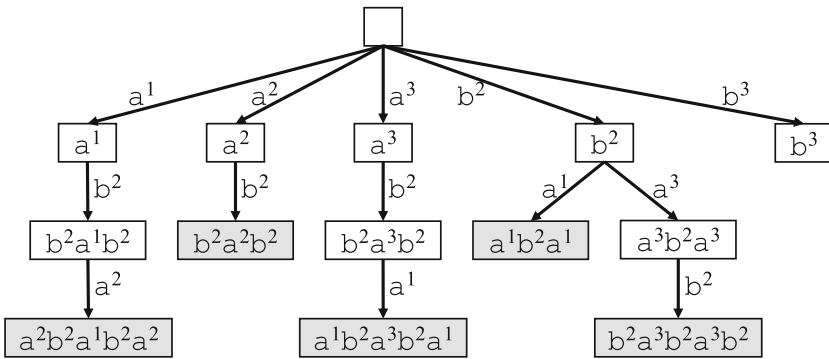


Fig. 2. The RLE-eertree $\text{e}^2\text{rtre}^2(S)$ of $RLE_S = b^3a^2b^2a^1b^2a^3b^2a^3b^3$. Each white node represents a run-centered palindromic substring of S that has an RLE-bounded occurrence, while each gray node represents a run-centered palindromic substring of S that has a maximal occurrence in S .

Lemma 5. Let S be any string of which the size of RLE_S is m . Then, the number of nodes in $\text{e}^2\text{rtre}^2(S)$ is at most $2m + 1$.

Proof. First, we consider RLE_S as a string of length m over the alphabet Σ_{RLE} . It now follows from Lemma 1 that the number of non-empty distinct run-centered palindromic substrings of S that have an RLE-bounded occurrence is at most m . Each of these palindromic substrings are represented by a node of $e^2rtre^2(S)$, and let $e^2rtre^2(S)'$ denote the tree consisting only of these nodes (in the example of Fig. 2, $e^2rtre^2(S)'$ is the tree consisting only of the white nodes).

Now we count the number of nodes in $e^2rtre^2(S)$ that do not belong to $e^2rtre^2(S)'$ (the gray nodes in the running example of Fig. 2). Since each palindrome represented by this type of node has a run-centered maximal occurrence in S , the number of such palindromes is bounded by the number m of runs in RLE_S .

Hence, including the root that represent the empty string, there are at most $2m + 1$ nodes in $e^2rtre^2(S)$. $\square$

Lemma 6. *Given RLE_S of size m , $e^2rtre^2(S)$ can be built in $O(m \log \sigma_{RLE})$ time and $O(m)$ space, where the out-going edges of each node are sorted according to the total order of their labels. Also, in the resulting $e^2rtre^2(S)$, each non-root node u stores the number of occurrences of u in S which are RLE-bounded or maximal.*

Proof. Our construction algorithm comprises three steps. We firstly build $e^2rtre^2(S)'$, secondly compute an auxiliary array $CPal$ that will be used for the next step, and thirdly we add some nodes that represent run-centered maximal palindromes which are not in $e^2rtre^2(S)'$ so that the resulting tree forms the final structure $e^2rtre^2(S)$.

Rubinchik and Shur [12] proposed an online algorithm to construct $eertree(T)$ of a string of length k in $O(k \log \sigma_T)$ time with $O(k)$ space, where σ_T denotes the number of distinct characters in T . They also showed how to store, in each node, the number of occurrences of the corresponding palindromic substring in T . Thus, the Rubinchik and Shur algorithm applied to RLE_S computes $e^2rtre^2(S)'$ in $O(m \log \sigma_{RLE})$ time with $O(m)$ space. Also, now each node u of $e^2rtre^2(S)'$ stores the number of RLE-bounded occurrence of u in S . This is the first step.

The second step is as follows: Let $CPal$ be an array of length m such that, for each $1 \leq i \leq m$, $CPal[i]$ stores a pointer to the node in $e^2rtre^2(S)'$ that represents the RLE-bounded palindrome centered at i . A simple application of the Rubinchik and Shur algorithm to RLE_S only gives us the leftmost occurrence of each RLE-bounded palindrome in $e^2rtre^2(S)'$. Hence, we only know the values of $CPal$ in the positions that are the centers of the leftmost occurrences of RLE-bounded palindromes. To compute the other values in $CPal$, we run Manacher's algorithm to RLE_S as in Sect. 3.2. Since Manacher's algorithm processes RLE_S in an online manner from left to right, and since we already know the leftmost occurrences of all RLE-bounded palindromes in RLE_S , we can copy the pointers from previous occurrences. In case where an RLE-bounded palindrome extends with a newly read RLE-character (a, e) after it is copied from a previous occurrence during the execution of Manacher's algorithm, then we traverse the edge labeled (a, e) from the current node of $e^2rtre^2(S)'$. By repeating this until the

mismatch is found in extension of the current RLE-bounded palindrome, we can find the corresponding node for this RLE-bounded palindrome. This way we can compute $CPal[i]$ for all $1 \leq i \leq m$ in $O(m \log \sigma_{RLE})$ total time with $O(m)$ total space.

In the third step, we add new nodes that represent run-centered maximal (but not RLE-bounded) palindromic substrings. For this sake, we again apply Manacher's algorithm to RLE_S , but in this case it is done as in Remark 1 of Sect. 3.2. With the help of $CPal$ array, we can associate each run (a_l, e_l) with the RLE-bounded palindromic substring that has the center in (a_l, e_l) . Let $Q_l = S[Rb[i]..Re[j]]$ denote this palindromic substring for (a_l, e_l) , where $1 \leq i \leq l \leq j \leq m$, and u_l the node that represents Q_l in $\mathbf{e}^2\mathbf{rtre}^2(S)'$. We first check whether $a_{i-1} = a_{j+1}$. If no, then Q_l does not extend from this run (a_l, e_l) , and if yes then we extend both ends of Q_l with $(a_{i-1}, \min\{e_{i-1}, e_{j+1}\})$. Assume w.l.o.g. that $e_{i-1} = \min\{e_{i-1}, e_{j+1}\}$. If there is no out-going edge of u_l with label (a_{i-1}, e_{i-1}) , then we create a new child of u_l with an edge labeled (a_{i-1}, e_{i-1}) . Otherwise, then let v be the existing child of u_l that represents $a_{i-1}^{e_{i-1}} u_l a_{i-1}^{e_{i-1}}$. We increase the number of occurrences of v by 1. This way, we can add all new nodes and we obtain $\mathbf{e}^2\mathbf{rtre}^2(S)$. Note that each node stores the number of RLE-bounded or maximal occurrence of the corresponding run-centered palindromic substring. It is easy to see that the second step takes a total of $O(m \log \sigma_{RLE})$ time and $O(m)$ space. $\square$

It is clear that for any character $a \in \Sigma$, there can be only one MUPS of form a^e . Namely, a^e is a MUPS iff e is the largest exponent for all runs of a 's in S and $occ_S(a^e) = 1$. Below, we consider other forms of MUPSs. Let P be a non-empty palindromic substring of string S that has a run-centered RLE-bounded occurrence. For any character $a \in \Sigma$, let $\mathbf{emax}$ and $\mathbf{esec}$ denote the largest and second largest positive integers such that $a^{\mathbf{emax}}Pa^{\mathbf{emax}}$ and $a^{\mathbf{esec}}Pa^{\mathbf{esec}}$ are palindromes that have run-centered RLE-bounded or maximal occurrences in S . If such integers do not exist, then let $\mathbf{emax} = \text{nil}$ and $\mathbf{esec} = \text{nil}$.

Observation 1. *There is at most one MUPS of form a^ePa^e in S . Namely,*

- (1) *The palindrome $a^{\mathbf{esec}+1}Pa^{\mathbf{esec}+1}$ is a MUPS of S iff $\mathbf{emax} \neq \text{nil}$, $\mathbf{esec} \neq \text{nil}$, and $occ_S(a^{\mathbf{emax}}Pa^{\mathbf{emax}}) = 1$.*
- (2) *The palindrome a^1Pa^1 is a MUPS of S iff $\mathbf{emax} \neq \text{nil}$, $\mathbf{esec} = \text{nil}$, and $occ_S(a^{\mathbf{emax}}Pa^{\mathbf{emax}}) = 1$.*
- (3) *There is no MUPS of form a^ePa^e with any $e \geq 1$ iff either $\mathbf{emax} = \text{nil}$, or $\mathbf{emax} \neq \text{nil}$ and $occ_S(a^{\mathbf{emax}}Pa^{\mathbf{emax}}) > 1$.*

Lemma 7. $\mathcal{M}_S$ can be computed in $O(m \log \sigma_{RLE})$ time and $O(m)$ space.

Proof. For each node u of $\mathbf{e}^2\mathbf{rtre}^2(S)$, let Σ_u be the set of characters a such that there is an out-going edge of u labeled by (a, e) with some positive integer e . Due to Observation 1, for each character in Σ_u , it is enough to check the out-going edges which have the largest and second largest exponents with character a . Since the edges are sorted, we can find all children of u that represent MUPSs

in time linear in the number of children of u . Hence, given $\mathbf{e}^2\mathbf{rtre}^2(S)$, it takes $O(m)$ total time to compute all MUPSs in S . $\mathbf{e}^2\mathbf{rtre}^2(S)$ can be computed in $O(m \log \sigma_{RLE})$ time and $O(m)$ space by Lemma 6.

What remains is how to sort the MUPSs in increasing order of their beginning positions. We associate each MUPS with the run where its center lies. Since each MUPS occurs in S exactly once and MUPSs do not nest (Lemma 2), each run cannot contain the centers of two or more MUPSs. We compute an array A of size m such that $A[j]$ contains the corresponding interval of the MUPS whose center lies in the j th run in RLE_S , if it exists. After computing A , we scan A from left to right. Since again MUPSs do not nest, this gives as the sorted list $\mathcal{M}_S$ of MUPSs. It is clear that this takes a total of $O(m)$ time and space. $\square$

4 SUPS Queries on RLE Strings

In this section, we present our algorithm for SUPS queries. Our algorithm is based on Inoue et al.'s algorithm [8] for SUPS queries on a plain string. The big difference is that the space that we are allowed for is limited to $O(m)$.

4.1 Data Structures

As was discussed in Sect. 3, we can compute the list $\mathcal{M}_S$ of all MUPSs of string S efficiently. We store $\mathcal{M}_S$ using the three following arrays:

- $M_{beg}[i]$: the beginning position of the i th MUPS in $\mathcal{M}_S$.
- $M_{end}[i]$: the ending position the i th MUPS in $\mathcal{M}_S$.
- $M_{len}[i]$: the length of the i th MUPS in $\mathcal{M}_S$.

Since the number of MUPSs in $\mathcal{M}_S$ is at most m (Corollary 1), the length of each array is at most m . In our algorithm, we use *range minimum queries* and *predecessor/successor queries* on integer arrays.

Let A be an integer array of length d . A range minimum query $\mathbf{RmQ}_A(i, j)$ returns one of $\arg \min_{i \leq k \leq j} \{A[k]\}$ for a given interval $[i, j]$ in A .

Lemma 8 (e.g. [2]). *We can construct an $O(d)$ -space data structure in $O(d)$ time for an integer array A of length d which can answer $\mathbf{RmQ}_A(i, j)$ in constant time for any query $[i, j]$.*

Let B be an array of d positive integers in $[1, N]$ in increasing order. The predecessor and successor queries on B are defined for any $1 \leq k \leq N$ as follows.

$$\begin{aligned} \text{Pred}_B(k) &= \begin{cases} \max\{i \mid B[i] \leq k\} & \text{if it exists,} \\ 0 & \text{otherwise.} \end{cases} \\ \text{Succ}_B(k) &= \begin{cases} \min\{i \mid B[i] \geq k\} & \text{if it exists,} \\ N + 1 & \text{otherwise.} \end{cases} \end{aligned}$$

Lemma 9 ([1]). *We can construct, in $O(d\sqrt{\log d / \log \log d})$ time, an $O(d)$ -space data structure for an array B of d positive integer in $[1, N]$ in increasing order which can answer $\text{Pred}_B(k)$ and $\text{Succ}_B(k)$ in $O(\sqrt{\log d / \log \log d})$ time for any query $k \in [1, N]$.*

4.2 Query Algorithm

Our algorithm simulates the query algorithm for a plain string [8] with $O(m)$ -space data structures. We summarize our algorithm below.

Let $[s, t]$ be a query interval such that $1 \leq s \leq t \leq n$. Firstly, we compute the number of MUPSs contained in $[s, t]$. This operation can be done in $O(\sqrt{\log m / \log \log m})$ by using $\text{Succ}_{M_{\text{beg}}}(s)$ and $\text{Pred}_{M_{\text{end}}}(t)$.

Let num be the number of MUPSs contained in $[s, t]$. If $\text{num} \geq 2$, then there is no SUPS for this interval (Corollary 1 of [8]). Suppose that $\text{num} = 1$. Let $S[i..j]$ be the MUPS contained in $[s, t]$. If $S[i - z..j + z]$ is a palindromic substring, then $S[i - z..j + z]$ is the only SUPS for $[s, t]$ where $z = \max\{i - s, t - j\}$. Otherwise, there is no SUPS for $[s, t]$ (Lemma 6 of [8]). Since this candidate has a run as the center, we can check whether $S[i - z..j + z]$ is a palindromic substring or not in constant time after computing all run-centered maximal palindromes. Suppose that $\text{num} = 0$ (this case is based on Lemma 7 of [8]). Let $p = \text{Pred}_{M_{\text{end}}}(t)$, $q = \text{Succ}_{M_{\text{beg}}}(s)$. We can check whether each of $S[M_{\text{beg}}[p] - t + M_{\text{end}}[p]..t]$ and $S[s..M_{\text{end}}[q] + M_{\text{beg}}[q] - s]$ is a palindrome or not. If so, the shorter one is a candidate of SUPSs. Let ℓ be the length of the candidates. Other candidates are the shortest MUPSs which contain the query interval $[s, t]$. If the length of these candidates is less than or equal to ℓ , we need to compute these candidates as SUPSs. We can compute these MUPSs by using range minimum queries on $M_{\text{len}}[p + 1, q - 1]$. Thus, we can compute all SUPSs in linear time w.r.t. the number of outputs (see [8] in more detail).

We conclude with the main theorem of this paper.

Theorem 1. *Given RLE_S of size m for a string S , we can compute a data structure of $O(m)$ space in $O(m(\log \sigma_{\text{RLE}} + \sqrt{\log m / \log \log m}))$ time so that subsequent SUPS queries can be answered in $O(\alpha + \sqrt{\log m / \log \log m})$ time, where σ_{RLE} denotes the number of distinct RLE-characters in RLE_S and α the number of SUPSs to report.*

Acknowledgments. This work was supported by JSPS KAKENHI Grant Numbers JP18K18002 (YN), JP17H01697 (SI), JP16H02783 (HB), and JP18H04098 (MT).

References

1. Beame, P., Fich, F.E.: Optimal bounds for the predecessor problem and related problems. *J. Comput. Syst. Sci.* **65**(1), 38–72 (2002)
2. Bender, M.A., Farach-Colton, M.: The LCA problem revisited. In: Gonnet, G.H., Viola, A. (eds.) *LATIN 2000*. LNCS, vol. 1776, pp. 88–94. Springer, Heidelberg (2000). https://doi.org/10.1007/10719839_9
3. Droubay, X., Justin, J., Pirillo, G.: Episturmian words and some constructions of de Luca and Rauzy. *Theor. Comput. Sci.* **255**(1–2), 539–553 (2001)
4. Ganguly, A., Hon, W., Shah, R., Thankachan, S.V.: Space-time trade-offs for finding shortest unique substrings and maximal unique matches. *Theor. Comput. Sci.* **700**, 75–88 (2017)

5. Hon, W.-K., Thankachan, S.V., Xu, B.: An In-place framework for exact and approximate shortest unique substring queries. In: Elbassioni, K., Makino, K. (eds.) ISAAC 2015. LNCS, vol. 9472, pp. 755–767. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48971-0_63
6. Hu, X., Pei, J., Tao, Y.: Shortest unique queries on strings. In: Moura, E., Crochemore, M. (eds.) SPIRE 2014. LNCS, vol. 8799, pp. 161–172. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-11918-2_16
7. İleri, A.M., Külekci, M.O., Xu, B.: Shortest unique substring query revisited. In: Kulikov, A.S., Kuznetsov, S.O., Pevzner, P. (eds.) CPM 2014. LNCS, vol. 8486, pp. 172–181. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-07566-2_18
8. Inoue, H., Nakashima, Y., Mieno, T., Inenaga, S., Bannai, H., Takeda, M.: Algorithms and combinatorial properties on shortest unique palindromic substrings. *J. Discrete Algorithms* **52–53**, 122–132 (2018)
9. Manacher, G.: A new linear-time “on-line” algorithm for finding the smallest initial palindrome of a string. *J. ACM* **22**, 346–351 (1975)
10. Mieno, T., Inenaga, S., Bannai, H., Takeda, M.: Shortest unique substring queries on run-length encoded strings. In: *Proceedings of MFCS 2016*, pp. 69:1–69:11 (2016)
11. Pei, J., Wu, W.C.H., Yeh, M.Y.: On shortest unique substring queries. *Proc. ICDE* **2013**, 937–948 (2013)
12. Rubinchik, M., Shur, A.M.: EERTRE: an efficient data structure for processing palindromes in strings. *Eur. J. Comb.* **68**, 249–265 (2018)
13. Tsuruta, K., Inenaga, S., Bannai, H., Takeda, M.: Shortest unique substrings queries in optimal time. In: Geffert, V., Preneel, B., Rován, B., Štuller, J., Tjoa, A.M. (eds.) *SOFSEM 2014*. LNCS, vol. 8327, pp. 503–513. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-04298-5_44